

ADVANCE FILE ALLOCATION SIMULATOR USING GUI

A MINI-PROJECT REPORT Submitted by

KAVIN SAINATH 231901018

RAMANITHARAN 231901040

VISHWANATH 231901062

*in partial fulfillment of the award of the
degree of*

**BACHELOR OF ENGINEERING IN
COMPUTER SCIENCE AND ENGINEERING
(CYBER SECURITY)**



**RAJALAKSHMI ENGINEERING COLLEGE
CHENNAI**

An Autonomous Institute

CHENNAI

MAY 2025

BONAFIDE CERTIFICATE

Certified that this project “**ADVANCED FILE ALLOCATION: USING GUI**” is the bonafide work of “**KAVIN SAINATH, RAMANITHARAN & VISHWANATH**” who carried out the project work under my supervision.

SIGNATURE

Ms.V.JANANEE

ASSISTANT PROFESSOR

Dept. of Computer Science and Engg,

Rajalakshmi Engineering College

Chennai

This mini project report is submitted for the viva voce examination to be held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

The Advanced File Allocation Simulator with GUI for Kali Linux is a comprehensive, interactive application designed to simulate and visualize advanced file allocation techniques used in modern operating systems. Tailored for the Kali Linux platform, the simulator supports a wide range of allocation strategies—including contiguous, linked, indexed, and multilevel indexed allocation—with real-time graphical representation of file storage and access paths. It serves as a powerful educational and diagnostic tool for students, developers, and cybersecurity professionals seeking deeper insights into file system internals. . Users can create, modify, and delete files while observing how each allocation method handles data placement and retrieval on a simulated disk structure. Logical and physical file paths are dynamically generated and visualized to demonstrate how operating systems navigate file data in various allocation schemes.

TABLE OF CONTENTS

1	Introduction	1
1.1	Introduction	1
1.2	Scope of the Work	1
1.3	Problem Statement	2
1.4	Aim and Objectives of the Project	3
2	File Allocation Techniques	4
2.1	Contiguous Allocation	4
2.2	Linked Allocation	4
2.3	Indexed Allocation	5
3	System Design and Implementation	6
3.1	System Architecture	6
3.2	Module Description	6

3.3	GUI Design Overview	6
3.4	Storage Structure (JSON)	6
4	Visualization and Performance Analysis	7
4.1	Disk Block Visualization	7
4.2	Simulation Logs	7
4.3	Comparison of Allocation Methods	7
5	Results and Observations	8
5.1	Code Implementation	8
6	Conclusion and Future Enhancement	18
7	References	19

1. INTRODUCTION

1.1 INTRODUCTION

The concept of file allocation is a fundamental aspect of operating systems, which determines how files are stored, accessed, and managed on physical storage devices. However, due to its inherent complexity and the invisible nature of storage management, it is often challenging for students and beginners to fully grasp these concepts through theoretical study alone.

This project, titled "Advanced File Allocation Simulator", aims to bridge that gap by developing an interactive educational tool using Python programming language along with the Tkinter library for the Graphical User Interface (GUI).

The simulator is designed to practically demonstrate three primary file allocation strategies used in operating systems: Contiguous Allocation, Linked Allocation, and Indexed Allocation.

By visualizing these techniques dynamically, users can gain a better understanding of how files are distributed across memory blocks, how space is utilized, how fragmentation occurs, and how different allocation methods impact system performance. This project thus serves as a valuable learning aid for students, educators, and software enthusiasts who wish to delve deeper into operating system principles.

1.2 SCOPE OF THE WORK

The scope of this project is extensive yet focused to ensure effective learning outcomes. The Advanced File Allocation Simulator provides a user-friendly GUI that enables users to input files, allocate space using different strategies, delete files, and observe the real-time changes occurring in memory.

Key features and functionalities included within the simulator are:

- Interactive Disk Block Visualization: Each disk block is color-coded and displayed graphically to represent free and occupied spaces.
- Multiple Allocation Techniques: Users can choose between contiguous, linked, and indexed methods for file storage simulation.
- Dynamic Memory Tracking: The system continuously updates and tracks memory utilization after each operation.
- File Deletion and Reallocation: Users can delete files and free up memory blocks, experiencing how deallocation affects storage.
- Persistent Storage Management: By leveraging JSON files, the simulator maintains disk status across sessions.
- Error Handling Mechanisms: User-friendly messages guide users when allocation or deallocation operations encounter issues.

Thus, the simulator not only models real-world storage behavior but also enhances comprehension through an engaging and intuitive interface.

1.3 PROBLEM STATEMENT

Understanding file allocation mechanisms within an operating system has always been a challenge because of its abstract nature. Typically, operating systems manage storage invisibly in the background without providing any visual feedback to the user regarding how space is being allocated or fragmented.

Students and developers often struggle with questions such as:

- How are files stored contiguously or non-contiguously?
- What happens when there is insufficient space?
- How does fragmentation affect storage efficiency?
- What trade-offs exist between access speed and storage utilization?

1.4 AIM AND OBJECTIVES OF THE PROJECT

The main aim of this project is to develop a simulator that demonstrates and contrasts the behavior of various file allocation techniques used in modern operating systems. Through practical experimentation and real-time feedback, users will be able to comprehend file allocation mechanisms far better than by studying theory alone.

The objectives of the project include:

- To Implement and Visualize File Allocation Techniques:
Simulate Contiguous, Linked, and Indexed file allocation strategies with dynamic disk visualization.
- To Provide a Graphical User Interface (GUI):
Build an intuitive, accessible GUI using Tkinter that allows users to interact with the simulation effortlessly.

2. FILE ALLOCATION TECHNIQUES

2.1 CONTIGUOUS ALLOCATION

Contiguous Allocation is one of the simplest and most straightforward methods used to allocate disk space to files. In this technique, a single, contiguous block of memory is allocated to a file. The file occupies a continuous section of memory, meaning the file's data is stored in adjacent blocks. This approach is highly efficient when it comes to accessing the file, as the entire file is stored in one place, minimizing the overhead associated with file retrieval.

However, contiguous allocation suffers from a major drawback: external fragmentation. As files are allocated and deallocated over time, free blocks are scattered across the disk. This fragmentation leads to inefficient use of available memory, making it difficult to find a large enough contiguous space for new files or file expansion. While the system might have enough total free space, it may not be able to allocate that space in a single contiguous region. As a result, this method may lead to poor disk utilization over time, especially when dealing with large numbers of small files.

2.2 LINKED ALLOCATION

Linked Allocation is a more flexible file allocation method that addresses the issue of fragmentation by using a different approach. Instead of storing a file's data in contiguous blocks, linked allocation stores the file's data in non-contiguous blocks scattered across the disk. Each block contains a pointer to the next block in the file, forming a linked list of blocks. This way, the file can span across any available blocks on the disk.

One of the main advantages of linked allocation is that it solves the problem of external fragmentation. Since the file's blocks are not required to be stored contiguously, the system can allocate blocks from anywhere on the disk as long as they are free. This results in better disk

space utilization because files can be allocated regardless of the available free space being fragmented across the disk.

However, the downside to this method is increased access time. Since each block of the file is linked to the next, the system needs to follow the pointers to access each part of the file. This introduces overhead, especially when accessing large files, as the system must traverse multiple pointers to access different blocks. The non-contiguous nature of the allocation can lead to slower file access, making this method less efficient for frequently accessed files.

To summarize, linked allocation provides a solution to fragmentation but sacrifices speed due to the additional pointer dereferencing required for file access.

2.3 INDEXED ALLOCATION

Indexed Allocation is a more balanced approach that combines the advantages of contiguous and linked allocation methods. In this technique, a special index block is used to store the addresses of all the blocks that are part of a file. Instead of storing the file in a contiguous section of memory or in random non-contiguous blocks, the file's index block acts as a reference, pointing to the locations of the file's data blocks scattered across the disk.

This method addresses both internal fragmentation (by allowing non-contiguous allocation) and external fragmentation (by organizing file block addresses in a central index).

However, indexed allocation has some overhead. It requires an additional block to store the index, which increases memory usage slightly. The size of the index block also depends on the number of blocks allocated to the file; if a file is very large, the index itself can grow in size, potentially impacting efficiency. Despite this, indexed allocation strikes a balance between performance and space utilization.

3. SYSTEM DESIGN & IMPLEMENTATION

3.1 SYSTEM ARCHITECTURE

The system consists of a GUI interface, an internal memory manager, a disk visualizer, and a JSON-based storage module.

3.2 MODULE DESCRIPTION

- GUI: Accepts file input, displays results, and hosts buttons.
- Disk Manager: Handles allocation and deallocation.
- Storage Module: Reads/writes to JSON file.
- Visualizer: Updates canvas with color-coded blocks.

3.3 GUI DESIGN OVERVIEW

Built using Tkinter, the GUI consists of input fields, action buttons, a canvas for visualization, and an output text area.

3.4 STORAGE STRUCTURE (JSON)

The state of the disk and files is saved and loaded using a JSON file named `file_system.json`.

4. VISUALIZATION & PERFORMANCE ANALYSIS

4.1 DISK BLOCK VISUALIZATION

Canvas shows 50 blocks, green for free and red for used. Updates in real-time after every action.

4.2 SIMULATION LOGS

Text area logs user actions including allocations, deletions, and file system status.

4.3 COMPARISON OF ALLOCATION METHODS

Each method has trade-offs in space efficiency and access time. Users can compare methods via real-time behavior and disk usage.

5. RESULTS & OBSERVATIONS

The simulator effectively demonstrated how each allocation strategy functions. It showed the impact of fragmentation, space constraints, and how errors occur and are handled in a file system.

5.1 code implementation:

```
import tkinter as tk

from tkinter import ttk, messagebox

import random

import json

import os


DISK_SIZE = 50 # Number of disk blocks


class FileAllocationGUI:

    def init(self, root):

        self.disk = ["Free"] * DISK_SIZE # Initialize disk

        self.files = {} # Store file allocations

        self.root = root
```

```
self.root.title("Advanced File Allocation Simulator")
```

```
self.root.geometry("900x600")
```

```
# Top Section for Inputs
```

```
top_frame = tk.Frame(root)
```

```
top_frame.pack(pady=10)
```

```
tk.Label(top_frame, text="File Name:").grid(row=0, column=0, padx=5, pady=5)
```

```
tk.Label(top_frame, text="File Size:").grid(row=1, column=0, padx=5, pady=5)
```

```
self.file_name_entry = tk.Entry(top_frame)
```

```
self.file_size_entry = tk.Entry(top_frame)
```

```
self.file_name_entry.grid(row=0, column=1, padx=5, pady=5)
```

```
self.file_size_entry.grid(row=1, column=1, padx=5, pady=5)
```

```
# Buttons for Allocation
```

```
button_frame = tk.Frame(root)
```

```
button_frame.pack(pady=10)
```

```
        ttk.Button(button_frame, text="Contiguous", command=self.contiguous_allocation).grid(row=0,
column=0, padx=5, pady=5)
```

```
        ttk.Button(button_frame, text="Linked", command=self.linked_allocation).grid(row=0, column=1,
padx=5, pady=5)
```

```
        ttk.Button(button_frame, text="Indexed", command=self.indexed_allocation).grid(row=0, column=2,
padx=5, pady=5)
```

```
        ttk.Button(button_frame, text="Delete File", command=self.delete_file).grid(row=0, column=3,
padx=5, pady=5)
```

```
# Disk Representation
```

```
self.canvas = tk.Canvas(root, width=700, height=150, bg="white")
```

```
self.canvas.pack(pady=20)
```

```
self.draw_disk()
```

```
# Output Display
```

```
self.output_text = tk.Text(root, height=10, width=90)
```

```
self.output_text.pack(pady=10)
```

```
ttk.Button(root, text="Show File System", command=self.show_files).pack(pady=5)
```

```
ttk.Button(root, text="Exit", command=root.quit).pack(pady=5)
```

```
self.load_data()
```

```
def draw_disk(self):
```

```
    """Updates the graphical disk representation."""
```

```
    self.canvas.delete("all")
```

```
    block_width = 14
```

```
    for i in range(DISK_SIZE):
```

```
        color = "green" if self.disk[i] == "Free" else "red"
```

```
        self.canvas.create_rectangle(i * block_width, 20, (i + 1) * block_width, 50, fill=color,
outline="black")
```

```
        self.canvas.create_text(i * block_width + 7, 35, text=str(i), font=("Arial", 8))
```

```
def update_output(self, message):
```

```
    """Display messages in the output text area."""
```

```
    self.output_text.insert(tk.END, message + "\n")
```

```
    self.output_text.see(tk.END)
```

```
def save_data(self):
```



```
"""Save file system data."""
```

```
data = {"disk": self.disk, "files": self.files}
```

```
with open("file_system.json", "w") as f:
```

```
    json.dump(data, f)
```

```
def load_data(self):
```

```
    """Load file system data."""
```

```
    if os.path.exists("file_system.json"):
```

```
        with open("file_system.json", "r") as f:
```

```
            data = json.load(f)
```

```
            self.disk = data["disk"]
```

```
            self.files = data["files"]
```

```
            self.draw_disk()
```

```
def contiguous_allocation(self):
```

```
    """Allocates a file using contiguous allocation."""
```

```
    name = self.file_name_entry.get()
```

```
    size = int(self.file_size_entry.get())
```

```
    if name in self.files:
```

```
messagebox.showerror("Error", "File already exists!")
```

```
return
```

```
for i in range(DISK_SIZE - size + 1):
```

```
    if all(self.disk[j] == "Free" for j in range(i, i + size)):
```

```
        for j in range(i, i + size):
```

```
            self.disk[j] = name
```

```
        self.files[name] = list(range(i, i + size))
```

```
        self.update_output(f"Contiguous: {name} -> Blocks {self.files[name]}")
```

```
        self.draw_disk()
```

```
        self.save_data()
```

```
        return
```

```
messagebox.showerror("Error", "Not enough contiguous space!")
```

```
def linked_allocation(self):
```

```
    """Allocates a file using linked allocation."""
```

```
    name = self.file_name_entry.get()
```

```
    size = int(self.file_size_entry.get())
```

```
    if name in self.files:
```

```

        messagebox.showerror("Error", "File already exists!")

    return

    free_blocks = [i for i, v in enumerate(self.disk) if v == "Free"]

    if len(free_blocks) < size:

        messagebox.showerror("Error", "Not enough space!")

    return

    allocated_blocks = random.sample(free_blocks, size)

    for block in allocated_blocks:

        self.disk[block] = name

    self.files[name] = allocated_blocks

    self.update_output(f"Linked: {name} -> Blocks {allocated_blocks}")

    self.draw_disk()

    self.save_data()

def indexed_allocation(self):

    """Allocates a file using indexed allocation."""

    name = self.file_name_entry.get()

    size = int(self.file_size_entry.get())

```

```
if name in self.files:
```

```
    messagebox.showerror("Error", "File already exists!")
```

```
    return
```

```
free_blocks = [i for i, v in enumerate(self.disk) if v == "Free"]
```

```
if len(free_blocks) < size + 1:
```

```
    messagebox.showerror("Error", "Not enough space!")
```

```
    return
```

```
index_block = free_blocks.pop(0)
```

```
data_blocks = random.sample(free_blocks, size)
```

```
self.disk[index_block] = name
```

```
for block in data_blocks:
```

```
    self.disk[block] = name
```

```
self.files[name] = {"index_block": index_block, "data_blocks": data_blocks}
```

```
self.update_output(f"Indexed: {name} -> Index {index_block} -> Blocks {data_blocks}")
```

```
self.draw_disk()
```

```
self.save_data()
```

```
def delete_file(self):
```

```
"""Deletes a file from the file system."""
```

```
name = self.file_name_entry.get()
```

```
if name not in self.files:
```

```
    messagebox.showerror("Error", "File not found!")
```

```
    return
```

```
blocks = self.files.pop(name)
```

```
if isinstance(blocks, dict):
```

```
    self.disk[blocks["index_block"]] = "Free"
```

```
    for b in blocks["data_blocks"]:
```

```
        self.disk[b] = "Free"
```

```
else:
```

```
    for b in blocks:
```

```
        self.disk[b] = "Free"
```

```
self.update_output(f"File {name} deleted successfully!")
```

```
self.draw_disk()
```

```
self.save_data()
```

```
def show_files(self):
```

```
"""Displays the current file allocations."""
```

```
self.output_text.delete("1.0", tk.END)
```

```
for name, blocks in self.files.items():
```

```
    self.update_output(f"{name}: {blocks}")
```

```
if name == "main":
```

```
    root = tk.Tk()
```

```
    app = FileAllocationGUI(root)
```

```
    root.mainloop()
```

6. CONCLUSION AND FUTURE ENHANCEMENT

This simulator bridges

the gap between theory and practice in file allocation. Future improvements include:

- File searching and sorting
- Defragmentation functionality
- Support for larger disk sizes
- More allocation strategies

7. REFERENCES

- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, "Operating System Concepts"
- Python Tkinter Documentation
- JSON File Handling in Python Tutorials